

SQL NEW QUESTION 2 — Event Average Finish Time

Database: ATHLETICS_RESULTS_DB

1. DATABASE + TABLE SCHEMA QUERIES

```
CREATE DATABASE ATHLETICS_RESULTS_DB;
USE ATHLETICS_RESULTS_DB;

CREATE TABLE events (
    event_id INT PRIMARY KEY,
    event_name VARCHAR(100)
);

CREATE TABLE athletes (
    athlete_id INT PRIMARY KEY,
    athlete_name VARCHAR(100)
);

CREATE TABLE results (
    result_id INT PRIMARY KEY,
    event_id INT,
    athlete_id INT,
    finish_time DECIMAL(5,2),
    status ENUM('Recorded','Disqualified','DNS')
);
```

Table Structure

Table	Column	Data Type	Key
events	event_id	INT	PK
events	event_name	VARCHAR(100)	
athletes	athlete_id	INT	PK
athletes	athlete_name	VARCHAR(100)	
results	result_id	INT	PK
results	event_id	INT	FK
results	athlete_id	INT	FK
results	finish_time	DECIMAL(5,2)	
results	status	ENUM('Recorded','Disqualified','DNS')	

2. INSERT RECORDS INTO TABLES

```
INSERT INTO events VALUES
(1, '100m Sprint'),
(2, '200m Sprint');
```

```
INSERT INTO athletes VALUES
(1, 'Alice'),
(2, 'Brian'),
(3, 'Carlos'),
(4, 'Diana');
```

```
INSERT INTO results VALUES
(201, 1, 1, 11.20, 'Recorded'),
(202, 1, 2, 10.95, 'Recorded'),
(203, 1, 3, 12.10, 'Recorded'),
(204, 1, 4, 11.50, 'Disqualified'),
(205, 2, 1, 22.50, 'Recorded'),
(206, 2, 2, 21.80, 'Recorded'),
(207, 2, 3, 21.80, 'Recorded'),
(208, 2, 4, 23.10, 'DNS');
```

3. RESULT QUERY

```

SELECT
    e.event_name,
    a.athlete_name,
    r.finish_time,
    ROUND(
        AVG(r.finish_time) OVER (
            PARTITION BY r.event_id
        ), 2
    ) AS avg_event_time
FROM results r
JOIN events e
    ON r.event_id = e.event_id
JOIN athletes a
    ON r.athlete_id = a.athlete_id
WHERE r.status = 'Recorded'
ORDER BY
    e.event_name,
    r.finish_time,
    a.athlete_name;

```

4. RESULT / EXPECTED OUTPUT

event_name	athlete_name	finish_time	avg_event_time
100m Sprint	Brian	10.95	11.42
100m Sprint	Alice	11.20	11.42
100m Sprint	Carlos	12.10	11.42
200m Sprint	Brian	21.80	22.03
200m Sprint	Carlos	21.80	22.03
200m Sprint	Alice	22.50	22.03

SQL NEW QUESTION 1 — Days Between Customer Orders

Database: CUSTOMERS_ORDERS_DB

1. DATABASE + TABLE SCHEMA QUERIES

```
CREATE DATABASE CUSTOMERS_ORDERS_DB;
USE CUSTOMERS_ORDERS_DB;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(10,2)
);
```

Table Structure

Table	Column	Data Type	Key
customers	customer_id	INT	PK
customers	customer_name	VARCHAR(100)	
orders	order_id	INT	PK
orders	customer_id	INT	FK
orders	order_date	DATE	
orders	amount	DECIMAL(10,2)	

2. INSERT RECORDS INTO TABLES

```
INSERT INTO customers VALUES
(1, 'Aarav'),
(2, 'Diya'),
(3, 'Kabir');

INSERT INTO orders VALUES
(501, 1, '2025-08-02', 800.00),
(506, 1, '2025-08-07', 1200.00),
(514, 1, '2025-08-19', 950.00),
(602, 2, '2025-08-04', 650.00),
(609, 2, '2025-08-13', 1050.00),
(701, 3, '2025-08-06', 1500.00),
(708, 3, '2025-08-16', 1100.00);
```

3. RESULT QUERY

```
SELECT
    c.customer_name,
    o.order_id,
    o.order_date,
    DATEDIFF(
        o.order_date,
        LAG(o.order_date) OVER (
            PARTITION BY o.customer_id
            ORDER BY o.order_date, o.order_id
        )
    ) AS days_since_previous_order
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
ORDER BY
    c.customer_name,
    o.order_date,
```

o.order_id;

4. RESULT / EXPECTED OUTPUT SHOWN IN PDF

customer_name	order_id	order_date	days_since_previous_order
Aditi	101	2025-11-07T00:00:00.000Z	NULL
Aditi	102	2025-11-11T00:00:00.000Z	4
Aditi	105	2025-11-15T00:00:00.000Z	4
Bharat	201	2025-11-14T00:00:00.000Z	NULL

Note: The PDF's Sample Data uses Aarav/Diya/Kabir with August 2025 orders, while its Expected Output image shows Aditi/Bharat with November 2025 orders. Both are reproduced separately above, exactly as shown in the source.